

Geospatial operations with Turf.js

MapLibre draws, Turf computes. Coverage areas, DF cross-fixes, geofences, track speeds and great-circle paths for our ADS-B, AIS, radio and HF data, plus the gotchas that bit us. Every example runs in npm test.

01 Where Turf fits

MapLibre renders GeoJSON. It doesn't measure anything. **Turf** is a library of pure functions on GeoJSON: features go in, numbers or new features come out, and you hand those to a MapLibre `<Source>`. There is no map, no DOM and no state, so everything Turf does can be unit tested.

Layer	Does	In this repo
Feeds	Raw messages: readsb JSON, AIS reports, DF bearings, HF spots	src/data/raw
Factories	Raw message → GeoJSON. Turf does the geometry	src/geo
Turf	Distances, bearings, circles, great circles, point-in-polygon	@turf/turf 7.4
MapLibre	Draws the GeoJSON. Styling, clicks, popups	src/map, src/examples

docs/examples/turf/01-where-it-fits.test.ts

```
it('range rings around a station: plain GeoJSON in, plain GeoJSON out', () => {
  const station = [11.582, 48.1351]; // [lng, lat], Munich
  const rings = featureCollection(
    [10, 25, 50].map((km) => circle(station, km, { steps: 64, properties: { km } })),
  );

  expectValidGeoJSON(rings); // hand it straight to <Source type="geojson" data={rings}>
  expect(rings.features.map((f) => f.properties.km)).toEqual([10, 25, 50]);
  expect(rings.features[0]!.geometry.coordinates[0]).toHaveLength(65); // closed ring
});
```

One package or many. Turf 7 is published as one package per function (`@turf/distance`, `@turf/circle`, ...), and `@turf/turf` re-exports all of them. We install `@turf/turf` and import by name. The package is ES modules with `"sideEffects": false`, so Vite only bundles the functions you import. In a small library that needs one or two functions, install just those packages (`npm install @turf/distance`); the API is the same.

docs/examples/turf/01-where-it-fits.test.ts

```
// One package with everything, imported by name so Vite can drop what you don't use:
import { circle, featureCollection } from '@turf/turf';
```

Coming from Turf 6? Turf 7 uses named exports (`import { distance } from '@turf/distance'`, not a default import) and the types from `@types/geojson`. `polygon()` now throws if a ring isn't closed. `buffer()` can return undefined for invalid input. In 7.4, `nearestPointOnLine` renamed its result properties (`index` → `segmentIndex`, `location` → `totalDistance`, `dist` → `pointDistance`). The old names still work but are deprecated.

02 Conventions

Five rules that every Turf call relies on:

Convention	What it means
<code>[lng, lat]</code>	Longitude first, always. A swapped position is still valid input, so nothing warns you.
GeoJSON in, GeoJSON out	Inputs take a Position, a Point or a Point Feature. Results are Features: read <code>.geometry.coordinates</code> .
<code>units</code>	Kilometres by default (<code>distance</code> , <code>length</code> , <code>along</code> , <code>destination</code> , <code>circle</code> , <code>buffer</code>). <code>area()</code> is always m ² . Use <code>'nauticalmiles'</code> for AIS and aviation.
Geodesic vs planar	<code>distance</code> , <code>bearing</code> , <code>destination</code> , <code>circle</code> , <code>greatCircle</code> , <code>length</code> and <code>area</code> work on a sphere (radius 6,371,008.8 m). <code>booleanPointInPolygon</code> , <code>lineIntersect</code> , <code>simplify</code> and <code>bbox</code> work on the raw numbers, as if lng/lat were flat.
Bearings	<code>bearing()</code> returns $-180...180$ (0 = north, clockwise). <code>destination()</code> documents $-180...180$ as input. <code>bearingToAzimuth</code> converts to the 0...360 a DF display shows.

Swapped coordinates are the most common bug, and Turf will not catch them:

`docs/examples/turf/02-conventions.test.ts`

```
it('positions are [lng, lat]; swapping them is valid input, just the wrong place', () => {
  const swapped = [munich[1]!, munich[0]!]; // [48.1351, 11.582]: in the Gulf of Aden
  expect(distance(munich, swapped)).toBeGreaterThan(5000); // km, and no error raised
});
```

`docs/examples/turf/02-conventions.test.ts`

```
it('accepts a Position, a Point geometry or a Point Feature, and returns GeoJSON', () => {
  const asFeature = point(munich, { stationId: 'rx-wst-01' });
  expect(distance(asFeature, hamburg)).toBe(distance(asFeature.geometry, hamburg));

  const end = destination(munich, 40, 90); // a Feature<Point>, not a Position
  expect(end.type).toBe('Feature');
  expect(end.geometry.coordinates[0]).toBeGreaterThan(munich[0]!);
});
```

docs/examples/turf/02-conventions.test.ts

```
it('kilometres unless you say otherwise', () => {
  expect(distance(munich, hamburg)).toBeCloseTo(612.4, 1);
  expect(distance(munich, hamburg, { units: 'nauticalmiles' })).toBeCloseTo(330.7, 1);
  expect(distance(munich, hamburg, { units: 'meters' })).toBeCloseTo(612_429, 0);
});
```

docs/examples/turf/02-conventions.test.ts

```
it('bearing() returns -180..180; compass bearings are 0..360', () => {
  const towardsAugsburg = bearing(munich, [10.8978, 48.3705]); // west-north-west
  expect(towardsAugsburg).toBeCloseTo(-62.4, 1);
  expect(bearingToAzimuth(towardsAugsburg)).toBeCloseTo(297.6, 1); // what a DF display shows
});
```

Geodesic vs planar matters as soon as shapes get big. A polygon edge is a straight line in lng/lat, not a great circle:

docs/examples/turf/02-conventions.test.ts

```
it('distances are great-circle, but polygon edges are straight lines in lng/lat', () => {
  // A strip between 40°N and 50°N, 200° of longitude wide.
  const strip = polygon([[[-100, 40], [100, 40], [100, 50], [-100, 50], [-100, 40]]]);
  // On the globe, the great circle from [-100, 50] to [100, 50] passes near 82°N,
  // so [0, 55] would be inside. booleanPointInPolygon follows the straight edge at 50°N:
  expect(booleanPointInPolygon([0, 55], strip)).toBe(false);
});
```

03 Function map

The functions we use or reach for, checked against the 7.4 type definitions. Distances are in kilometres unless `units` says otherwise.

Measurement

Function	Returns	Notes
<code>distance(a, b, { units })</code>	number	Great-circle (haversine) distance
<code>bearing(a, b, { final })</code>	number, -180...180	Initial bearing; <code>final: true</code> for the bearing on arrival
<code>destination(origin, dist, bearing, { units })</code>	<code>Feature<Point></code>	Where you end up. Doesn't wrap longitudes (§05)
<code>length(line, { units })</code>	number	Sum of the segment distances
<code>area(polygon)</code>	number, m ²	Geodesic; holes are subtracted

Function	Returns	Notes
<code>along(line, dist, { units })</code>	<code>Feature<Point></code>	The point <code>dist</code> along a line
<code>midpoint(a, b)</code>	<code>Feature<Point></code>	Halfway along the great circle, not the lng/lat average
<code>nearestPoint(target, points)</code>	<code>Feature<Point></code>	Adds <code>featureIndex</code> and <code>distanceToPoint</code>

Creation

Function	Returns	Notes
<code>point</code> , <code>lineString</code> , <code>polygon</code>	<code>Feature</code>	Validate their input; <code>polygon</code> needs closed rings of ≥ 4 positions
<code>circle(center, radius, { steps, units })</code>	<code>Feature<Polygon></code>	64 steps by default
<code>bbox(geojson)</code>	<code>[w, s, e, n]</code>	Min/max of the raw numbers (\$05)
<code>bboxPolygon(bbox)</code>	<code>Feature<Polygon></code>	Rectangle from a bbox

Booleans

Function	Returns	Notes
<code>booleanPointInPolygon(pt, poly, { ignoreBoundary })</code>	<code>boolean</code>	Holes respected; on the boundary counts as inside by default
<code>booleanIntersects(a, b)</code>	<code>boolean</code>	True when the two geometries share any point
<code>booleanValid(f)</code>	<code>boolean</code>	OGC Simple Features validity (self-intersections and so on)
<code>booleanClockwise(ring)</code>	<code>boolean</code>	RFC 7946: outer rings are counter-clockwise

Transformation

Function	Returns	Notes
<code>buffer(geojson, radius, { units, steps })</code>	<code>Feature</code> or <code>undefined</code>	8 steps by default. <code>undefined</code> for invalid input
<code>simplify(geojson, { tolerance, highQuality })</code>	same type	Douglas-Peucker in degrees, not km

Function	Returns	Notes
<code>rewind(geojson, { reverse })</code>	same type	Fixes winding: outer CCW, holes CW
<code>truncate(geojson, { precision, coordinates })</code>	same type	6 decimals and 3 coordinates by default; a copy unless <code>mutate</code>
<code>cleanCoords(geojson)</code>	same type	Drops duplicate and collinear points

Lines

Function	Returns	Notes
<code>lineIntersect(a, b)</code>	<code>FeatureCollection<Point></code>	Planar; densify long lines first (§04)
<code>nearestPointOnLine(line, pt)</code>	<code>Feature<Point></code>	<code>segmentIndex</code> , <code>totalDistance</code> , <code>pointDistance</code> (7.4 names)
<code>lineSlice(start, stop, line)</code>	<code>Feature<LineString></code>	The part of a line between two points
<code>greatCircle(a, b, { npoints, offset })</code>	LineString or MultiLineString	100 points by default. Read §04 and §05 before you use it

04 Domain recipes

Coverage area from a station

`circle()` gives a polygon with 64 points, which is a fine first model of a receiver's range. Swap in a terrain-aware model later; the polygon's shape on the map stays the same. `coverageToFeature` wraps it with rounding and properties:

`docs/examples/turf/04-recipes.test.ts`

```
it('circle() gives a 64-sided polygon; our factory adds rounding and properties', () => {
  const raw = circle(munichRx, 28, { steps: 64, units: 'kilometers' });
  const ours = coverageToFeature(munichRx, 28, 'rx-wst-01');

  expect(ours.geometry.coordinates[0]).toHaveLength(raw.geometry.coordinates[0].length);
  expect(ours.properties).toEqual({ stationId: 'rx-wst-01', kind: 'coverage', rangeKm: 28 });
  expect(booleanPointInPolygon([11.9, 48.2], ours)).toBe(true); // ~25 km east: covered
  expect(booleanPointInPolygon([12.0, 48.2], ours)).toBe(false); // ~32 km east: not covered
});
```

Near ± 180 , Turf keeps the ring continuous, so longitudes run past 180. MapLibre draws that without a seam. Point-in-polygon tests are planar, though, so a contact reported at -179.95 has to be shifted by 360° before it counts as inside:

docs/examples/turf/04-recipes.test.ts

```
it('near  $\pm 180$  the ring stays continuous, so longitudes run past 180', () => {
  const fiji = coverageToFeature([179.9, -17], 50, 'rx-fj');
  const lngs = fiji.geometry.coordinates[0]!.map((lng) => lng!);
  expect(Math.max(...lngs)).toBeGreaterThan(180); // MapLibre draws this without a seam

  // ...but a contact reported at -179.95 is not "inside" until you shift it by 360°
  expect(booleanPointInPolygon([-179.95, -17], fiji)).toBe(false);
  expect(booleanPointInPolygon([-179.95 + 360, -17], fiji)).toBe(true);
});
```

DF bearing line and a cross-fix

A bearing line is the station plus `destination()` along the bearing. `dfBearingToFeature` normalises the bearing to $0...360$ for display and converts it to $-180...180$ for Turf:

docs/examples/turf/04-recipes.test.ts

```
it('a bearing line is the station plus destination()', () => {
  const line = dfBearingToFeature(munichRx, 22.8, 40, { stationId: 'rx-wst-01' });
  const [start, end] = line.geometry.coordinates as [Position, Position];
  expect(start).toEqual(munichRx);
  expect(distance(start, end)).toBeCloseTo(40, 3); // km; rounding to 6 decimals costs cm
});
```

Two bearings from two stations cross at the transmitter. `lineIntersect` treats each segment as a straight line in lng/lat, so make each bearing a great-circle ray with enough points to follow the curve:

docs/examples/turf/cross-fix.ts

```
/**
 * Where two DF bearings cross. Each bearing becomes a great-circle ray `rangeKm` long,
 * densified so lineIntersect (which works on straight lng/lat segments) follows the curve.
 * Returns null when the rays don't cross within range.
 */
export function crossFix(a: DfBearing, b: DfBearing, rangeKm = 500): Feature<Point> | null {
  const ray = ({ station, bearingDeg }: DfBearing) => {
    const end = destination(station, rangeKm, azimuthToBearing(bearingDeg)); // -> [-180, 180]
    return greatCircle(station, end, { npoints: 100 });
  };
  return lineIntersect(ray(a), ray(b)).features[0] ?? null;
}
```

docs/examples/turf/04-recipes.test.ts

```
it('two bearings cross at the transmitter', () => {
  // Bearings measured at Munich and Augsburg to a transmitter near Freising [11.75, 48.40]
  const fix = crossFix(
    { station: munichRx, bearingDeg: 22.82 },
    { station: [10.8978, 48.3705], bearingDeg: 86.70 },
    100,
  );
  expect(distance(fix, [11.75, 48.4])).toBeLessThan(0.1); // km
});
```

```
});

it('long HF bearings need densified rays: two-point lines miss by hundreds of km', () => {
  // Edinburgh and Cyprus taking bearings on a transmitter near Warsaw [21.0, 52.2]
  const edinburgh = { station: [-3.2, 55.95], bearingDeg: 94.73 };
  const cyprus = { station: [33.4, 35.2], bearingDeg: 336.36 };
  expect(distance(crossFix(edinburgh, cyprus, 3000!), [21.0, 52.2])).toBeLessThan(1);
  expect(distance(naiveCrossFix(edinburgh, cyprus, 3000!), [21.0, 52.2])).toBeGreaterThan(100);
});
```

Two-point lines are fine for VHF, wrong for HF. For bearings a few tens of km long, a straight line in lng/lat is close enough (the Freising fix is within 100 m). At HF ranges the same shortcut misses by hundreds of km. MapLibre also draws a two-point LineString as a straight line on the Mercator map, so draw long bearings from the densified ray as well.

docs/examples/turf/cross-fix.ts

```
/** The tempting version: two-point lines. Fine for short VHF bearings, not for HF. */
export function naiveCrossFix(a: DfBearing, b: DfBearing, rangeKm = 500) {
  const ray = ({ station, bearingDeg }: DfBearing) => {
    const end = destination(station, rangeKm, azimuthToBearing(bearingDeg));
    return lineString([station, end.geometry.coordinates]);
  };
  return lineIntersect(ray(a), ray(b)).features[0] ?? null;
}
```

Geofence alerts for AIS, including holes

`featuresInZone` is one line of Turf. The work is in the zone: the outer ring is the anchorage, and the hole is a cable area where anchoring is forbidden. `booleanPointInPolygon` respects holes:

src/geo/zones.ts

```
export function featuresInZone<P extends GeoJsonProperties>({
  features: Feature<Point, P>[],
  zone: Feature<Polygon | MultiPolygon>,
}): Feature<Point, P>[] {
  return features.filter((f) => booleanPointInPolygon(f, zone));
}
```

docs/examples/turf/04-recipes.test.ts

```
it('alerts for vessels in the anchorage, not in the cable-area hole', () => {
  const vessels = [
    report(244_660_001, 4.0, 51.97), // in the anchorage
    report(244_660_002, 4.025, 51.98), // in the hole (cable area)
    report(244_660_003, 4.2, 51.97), // outside
  ].map(aisToFeature).filter(notNull);

  const inside = featuresInZone(vessels, zone); // booleanPointInPolygon under the hood
  expect(inside.map((v) => v.properties.mmsi)).toEqual([244_660_001]);
});

it('decide what "on the boundary" means', () => {
```

```
const onEdge = point([3.98, 51.97]); // exactly on the western edge
expect(booleanPointInPolygon(onEdge, zone)).toBe(true);
expect(booleanPointInPolygon(onEdge, zone, { ignoreBoundary: true })).toBe(false);
});
```

For entry and exit alerts, run this on each new position and compare with the vessel's previous state. Decide once whether "on the boundary" counts as inside, and write it in a test.

Track length and average speed from ADS-B

`length()` sums the great-circle distances between consecutive points. Dividing by the time between the first and last kept sample gives the average ground speed. Speeds between consecutive samples are a cheap check for bad positions:

docs/examples/turf/04-recipes.test.ts

```
it('length() of the track, and speed from consecutive samples', () => {
  const track = adsbTrackToFeature(trackRaw.icao24, trackRaw.samples!);
  const km = length(track); // kilometres by default
  const { startTime, endTime } = track.properties;
  const hours = (Date.parse(endTime) - Date.parse(startTime)) / 3_600_000;
  const avgKt = km / 1.852 / hours;

  expect(km).toBeCloseTo(13.3, 1);
  expect(avgKt).toBeGreaterThan(200); // a departure climbing out of EDDM
  expect(avgKt).toBeLessThan(300);

  // Segment speeds: the same samples, pairwise. A spike here means a bad position.
  const s = trackRaw.samples;
  const segmentKt = s.slice(1).map((b, i) => {
    const a = s[i!];
    const dtH = (Date.parse(b.t) - Date.parse(a.t)) / 3_600_000;
    return distance([a.lon, a.lat], [b.lon, b.lat], { units: 'nauticalmiles' }) / dtH;
  });
  expect(Math.max(...segmentKt)).toBeLessThan(400);
});
```

Nearest vessel or aircraft to a point

`nearestPoint` returns a copy of the closest feature, with its index and distance added to the properties. It checks every feature, which is fine for hundreds of contacts (§05 covers thousands):

docs/examples/turf/04-recipes.test.ts

```
it('nearestPoint() returns the closest feature plus its index and distance', () => {
  const contacts = featureCollection([
    point([4.0, 51.97], { mmsi: 244_660_001 }),
    point([4.3, 52.1], { mmsi: 244_660_004 }),
    point([3.5, 51.8], { mmsi: 244_660_005 }),
  ]);
  const pilotBoat = [4.05, 51.98];
  const nearest = nearestPoint(pilotBoat, contacts);

  expect(nearest.properties.mmsi).toBe(244_660_001);
  expect(nearest.properties.featureIndex).toBe(0);
  expect(nearest.properties.distanceToPoint).toBeCloseTo(3.6, 1); // km
});
```

HF great-circle paths, and why we normalise Turf's output

A great circle is the shortest path, and on a Mercator map it curves. `greatCircle` densifies it so it draws correctly. It splits at the antimeridian *sometimes*, depending on `npoints` and `offset`. It can also end a path at +180 after approaching from the west, and return a part with a single point.

`hfPathToFeature` therefore always flattens Turf's output and runs it through `splitAtAntimeridian` (both bugs are in the fast-check guide, §07):

docs/examples/turf/04-recipes.test.ts

```
it('Turf splits at the antimeridian only sometimes; our factory always does', () => {
  const tokyo = [139.69, 35.69];
  const sanFrancisco = [-122.42, 37.77];
  expect(greatCircle(tokyo, sanFrancisco, { npoints: 8 }).geometry.type).toBe('LineString');
  expect(greatCircle(tokyo, sanFrancisco).geometry.type).toBe('MultiLineString'); // 100 points

  const path = hfPathToFeature(spot(tokyo, sanFrancisco), 8!);
  expectValidGeoJSON(path);
  expect(path.geometry.type).toBe('MultiLineString'); // split exactly at ±180, no gap
  expect(path.properties.distanceKm).toBe(8271);
});
```

src/geo/hf.ts

```
/**
 * Great-circle path between tx and rx, densified so it curves correctly on a Mercator map.
 * Paths that cross the antimeridian come back as a MultiLineString split exactly at ±180.
 * Returns null when tx and rx are (nearly) the same point, or (nearly) antipodal,
 * where the great circle is undefined.
 *
 * Known limit (found by the fast-check tests): a path that passes over or close to a pole
 * flips longitude by 180° at the pole. Web Mercator can't show latitudes beyond ~85°,
 * so such paths look odd on any Mercator map; use a globe projection if you need them.
 */
export function hfPathToFeature(
  spot: HfSpot,
  npoints = 64,
): Feature<LineString | MultiLineString, HfPathProps> | null {
  const [txLng, txLat] = spot.tx as [number, number];
  const [rxLng, rxLat] = spot.rx as [number, number];
  if (!isValidLngLat(txLng, txLat) || !isValidLngLat(rxLng, rxLat)) return null;

  const distanceKm = distance([txLng, txLat], [rxLng, rxLat], { units: 'kilometers' });
  const HALF_EARTH_KM = 20015; // half the circumference: antipodal
  if (distanceKm < MIN_PATH_KM || distanceKm > HALF_EARTH_KM - 100) return null;

  const g = greatCircle([txLng, txLat], [rxLng, rxLat], { npoints }).geometry;
  const flat = g.type === 'LineString' ? g.coordinates : g.coordinates.flat();
  const parts = splitAtAntimeridian(flat);
  if (parts.length === 0) return null;
  const geometry: LineString | MultiLineString =
    parts.length === 1
      ? { type: 'LineString', coordinates: parts[0]! }
      : { type: 'MultiLineString', coordinates: parts };

  return {
    type: 'Feature',
    geometry,
    properties: {
      kind: 'path',
      txCall: spot.txCall,
    },
  };
}
```

```

    rxCall: spot.rxCall,
    frequencyKHz: spot.frequencyKHz,
    band: bandForKHz(spot.frequencyKHz),
    mode: spot.mode,
    snrDb: spot.snrDb,
    distanceKm: Math.round(distanceKm),
    timestamp: spot.timestamp,
  },
};
}

```

05 Gotchas

Gotcha	What happens	What we do
Antimeridian	<code>circle</code> and <code>destination</code> return longitudes past ± 180 . <code>bbox</code> of a line across ± 180 spans the world the other way. <code>greatCircle</code> splits inconsistently	Normalise with <code>splitAtAntimeridian</code> . Shift by 360° for point-in-polygon tests near ± 180
Antipodes	<code>greatCircle</code> throws for antipodal points, where every great circle is equally short	<code>hfPathToFeature</code> returns <code>null</code> within 100 km of antipodal
Poles	A circle around a polar station doesn't contain the pole. Great circles over a pole flip longitude by 180°	Web Mercator stops at about 85° . Use the globe projection for polar work
Large scales	<code>buffer</code> is exact around a point, approximate along a long line. <code>lineIntersect</code> and <code>bbox</code> are planar	Densify, prefer geodesic functions, and check the error at your scale in a test
Precision	Results carry floating-point noise and <code>-0</code>	<code>truncate</code> (or our <code>round6</code>) before comparing or snapshotting
Many features	Turf functions check every feature you pass them	Index first: Turf's <code>geojsonRbush</code> , or <code>kdbush</code> / <code>geokdbush</code> for points

The antimeridian. -180 and $+180$ are the same meridian, but Turf mostly does arithmetic on the numbers:

`docs/examples/turf/05-gotchas.test.ts`

```

it('destination() does not wrap longitudes', () => {
  const end = destination([179.9, 10], 50, 90).geometry.coordinates;
  expect(end[0]).toBeGreaterThan(180); // 180.357, not -179.643
});

it('bbox() of a line across ±180 spans the whole world the other way', () => {
  const crossing = lineString([[170, 0], [-170, 1]]); // 20° wide on the globe
  expect(bbox(crossing)).toEqual([-170, 0, 170, 1]); // 340° wide as a bbox
});

```

Near the poles. A ring in lng/lat can't go *around* a pole, so the pole ends up outside:

docs/examples/turf/05-gotchas.test.ts

```
it('a coverage circle around a polar station does not contain the pole', () => {
  const ring = circle([0, 89.5], 100, { steps: 64 }); // the pole is 56 km away
  expect(booleanPointInPolygon([0, 89.5], ring)).toBe(true);
  expect(booleanPointInPolygon([0, 90], ring)).toBe(false); // geographically it should be true
});
```

Accuracy at large scales. `buffer` projects around the feature's centre, so it is exact for a point and drifts along a long line. `area` is geodesic, so a square degree at 60° N is about half the size of one at the equator:

docs/examples/turf/05-gotchas.test.ts

```
it('buffer() is exact around a point, approximate along a long line', () => {
  const line = lineString([[ -30, 60], [30, 60]]); // 3,300 km across the North Atlantic
  const zone = buffer(line, 100) as Feature<Polygon>; // km
  const gaps = zone.geometry.coordinates[0]!.map((p) => pointToLineDistance(p, line));
  expect(Math.min(...gaps)).toBeLessThan(99); // up to ~1 km short of the requested 100 km
});

it('area() is geodesic: one square degree shrinks with latitude', () => {
  const atEquator = area(bboxPolygon([0, 0, 1, 1])); // m²
  const at60North = area(bboxPolygon([0, 60, 1, 61]));
  expect(at60North / atEquator).toBeCloseTo(0.49, 2);
});
```

Precision. Turf returns full doubles. Truncate before you snapshot or compare, or a refactor that changes the 16th digit breaks your tests:

docs/examples/turf/05-gotchas.test.ts

```
it('results carry floating-point noise; truncate() before snapshots or comparisons', () => {
  const route = lineString([[0, 0], [2, 0], [4, 0]]);
  const leg = lineSlice([1, 0.1], [3, -0.1], route);
  expect(leg.geometry.coordinates.at(-1)).toEqual([3.0000000000000004, -0]); // noise and -0

  const clean = truncate(leg, { precision: 6 }); // returns a copy unless { mutate: true }
  expect(clean.geometry.coordinates).toEqual([[1, 0], [2, 0], [3, 0]] satisfies Position[]);
});
```

Many features. `nearestPoint`, `booleanPointInPolygon` over a list and `lineIntersect` on big collections check every feature. That is fine for one AIS feed area. For a whole region's ADS-B every second, build an index once per update, query it with a bounding box, and run Turf only on the candidates. Turf includes `geojsonRbush`, an R-tree over GeoJSON features:

docs/examples/turf/05-gotchas.test.ts

```
it('an R-tree narrows 10,000 contacts to a handful before Turf looks at them', () => {
  // A 100 x 100 grid of contacts over the North Sea and around it.
  const contacts = featureCollection(
    Array.from({ length: 10_000 }, (_, i) =>
      point([-10 + (i % 100) * 0.3, 45 + Math.floor(i / 100) * 0.15], { id: i })),
  );
  const index = geojsonRbush<Point, { id: number }>().load(contacts); // once per update
```

```
const zone = circle([4, 52], 25); // km
const candidates = index.search(zone).features; // bbox overlap only: cheap
const inside = candidates.filter((c) => booleanPointInPolygon(c, zone)); // exact: Turf

expect(candidates.length).toBeLessThan(20);
expect(inside).toHaveLength(
  contacts.features.filter((c) => booleanPointInPolygon(c, zone)).length, // the slow way
);
});
```

For points only, **kdbush** is a faster static index, and **geokdbush** adds nearest-neighbour queries on the sphere that handle the date line. Neither is a dependency of this repo yet.

06 Testing Turf-based code

Geometry code is where property-based testing pays off most: the edge cases (± 180 , the poles, antipodes, zero length) are exactly the inputs nobody writes by hand. The **fast-check onboarding guide** (<docs/fast-check-onboarding.pdf>) covers the tooling. Three patterns are specific to geometry:

1. Use Turf as an oracle, through different functions. If the code under test builds a circle with `circle()`, check it with `booleanPointInPolygon` and `distance`, not by calling `circle()` again:

test/radio.test.ts

```
test.prop([arbSafePosition, arbRange])(
  'contains its station and every vertex is ~rangeKm away',
  (station, rangeKm) => {
    const f = coverageToFeature(station, rangeKm, 'x');
    expectValidGeoJSON(f);
    expect(booleanPointInPolygon(point(station), f)).toBe(true);
    for (const v of f.geometry.coordinates[0]!) {
      expect(distance(station, v, { units: 'kilometers' })).toBeCloseTo(rangeKm, 0);
    }
  },
);
```

An oracle can also be a plain geometric fact. A track can never be shorter than the straight line between its ends. Our first version compared the two with an absolute tolerance of 10^{-9} km. fast-check found a track that runs out to the antipode, where Turf's haversine rounding made the track 0.65 mm *shorter* than its end-to-end distance. Comparing two computed distances needs a relative tolerance:

docs/examples/turf/06-testing.test.ts

```
// Oracle: the straight-line (great-circle) distance can never exceed the flown distance.
const t = '2020-01-01T00:00:00.000Z';
const toTheAntipode = [
  { lon: -179.99999850000003, lat: 0, t },
  { lon: 0, lat: 2.75e-5, t },
  { lon: 0, lat: 0, t },
];
// fast-check found that one: haversine loses precision near the antipode
test.prop([arbTrackSamples], { examples: [[toTheAntipode]] })(
  'a track is never shorter than its end-to-end distance',
```

```
(s) => {
  const track = adsbTrackToFeature('3c6a4f', s);
  if (!track) return;
  const c = track.geometry.coordinates;
  const straight = distance(c[0]!, c.at(-1)!);
  expect(length(track)).toBeGreaterThanOrEqual(straight * (1 - 1e-9)); // relative tolerance
},
);
```

2. Build the scenario backwards. Choose the answer first (the transmitter), derive the inputs from it (bearings measured with `bearing()`), then check that the code finds the answer again. This property checks `crossFix` between 60° S and 60° N, at ranges from 20 to 1,500 km:

docs/examples/turf/06-testing.test.ts

```
// Build the scenario backwards: place a transmitter, put two stations around it,
// measure the true bearings with Turf, then check that the fix lands on the transmitter.
const arbScenario = fc.record({
  tx: fc.tuple(
    fc.double({ min: -150, max: 150, noNaN: true }),
    fc.double({ min: -60, max: 60, noNaN: true }),
  ),
  bearingA: fc.double({ min: -180, max: 180, noNaN: true }), // from tx to station A
  turn: fc.double({ min: 30, max: 150, noNaN: true }), // angle between the two stations
  kmA: fc.double({ min: 20, max: 1500, noNaN: true }),
  kmB: fc.double({ min: 20, max: 1500, noNaN: true }),
});

test.prop([arbScenario])('the cross-fix lands within 1 km of the transmitter', (s) => {
  const place = (deg: number, km: number): Position =>
    destination(s.tx, km, deg).geometry.coordinates;
  const a = place(s.bearingA, s.kmA);
  const b = place(s.bearingA + s.turn, s.kmB);
  const towardsTx = (station: Position) => bearingToAzimuth(bearing(station, s.tx));

  const fix = crossFix(
    { station: a, bearingDeg: towardsTx(a) },
    { station: b, bearingDeg: towardsTx(b) },
    2000,
  );
  expect(fix).not.toBeNull();
  expect(distance(fix!, s.tx)).toBeLessThan(1);
});
```

3. Pin Turf's quirks. The tests in §04 and §05, and the case study in the fast-check guide, assert what Turf does today: longitudes past 180, inconsistent splitting, the single-point part and the `+180` endpoint. When a Turf upgrade changes any of them, a test fails and tells you which of our workarounds to revisit.

Use `arbPosition` (whole globe) by default. Switch to `arbSafePosition` only for models that are known to break at the poles or the antimeridian, and write a separate test that documents that limit.

07 Exercises and bookmarks

1. **Range rings.** Write `rangeRings(station, kms)` returning a `FeatureCollection`. Property: every ring contains the station and every vertex of the smaller rings.
2. **Closest point of approach.** For two AIS vessels with SOG and COG, project both positions every minute for 30 minutes with `destination()` and return the minimum distance and when it occurs. Test it with two vessels on a collision course.
3. **Entry and exit events.** From a sequence of positions for one MMSI, emit `enter` and `exit` events for the anchorage zone. Test with a track that crosses the cable-area hole.
4. **Geofence across ± 180 .** Make `featuresInZone` work for a zone that crosses the antimeridian. Write the failing test first (the coverage recipe in §04 shows the problem).
5. **Three stations.** Cross-fix three DF bearings pairwise and return the centroid of the triangle plus its area as an error estimate (`centroid`, `area`).
6. **Simplify a track.** Simplify a long ADS-B track for display with `simplify`. Property: every original sample is within the tolerance of the simplified line. Watch the units: `tolerance` is in degrees.

Bookmarks

- Turf API reference (every function, with live examples): `turfjs.org`
- Units Turf accepts: `turfjs.org/docs/api/types/Units`
- Turf releases (breaking changes per version): `github.com/Turfjs/turf/releases`
- RFC 7946, GeoJSON (§3.1.6 winding, §3.1.9 antimeridian cutting): `datatracker.ietf.org/doc/html/rfc7946`
- The formulas behind distance, bearing and destination: `movable-type.co.uk/scripts/latlong.html`
- Spatial indexes: `github.com/mourner/rbush`, `github.com/mourner/kdbush`, `github.com/mourner/geokdbush`

Sources

- Signatures, defaults and units: the installed type definitions, `node_modules/@turf/*/dist/esm/index.d.ts` (Turf 7.4.0)
- Behaviour (antimeridian, poles, planar intersections, precision): demonstrated by `docs/examples/turf/*.test.ts`, which run in `npm test`
- Turf 6 → 7 changes: Turf changelog, 7.0.0 — `github.com/Turfjs/turf/blob/master/CHANGELOG.md`
- `nearestPointOnLine` property names: the deprecation notes in `node_modules/@turf/nearest-point-on-line/dist/esm/index.d.ts`
- `geokdbush` features: its README — `github.com/mourner/geokdbush`